

AI伴学与职业成长平台 — Web管理端开发手册

本手册旨在指导项目开发团队在已具备 Vue 3 基础的前提下，利用 AI 辅助编程工具（如 Qoder、Trae 等）高效、规范地完成管理后台（Vue 3 + Vite + Element Plus + ECharts）的静态页面构建、架构分层，并为后续的前后端接口联调留出标准扩展空间。

一、项目模板创建（准备工作）

1. 概念引入：为什么需要手动初始化？

在开发任何基于单页面架构（SPA）的前端应用前，需要通过构建工具（Vite）初始化底层脚手架。这能确保基本的构建脚本（vite.config.js）、运行环境、包管理机制以及开发服务器配置处于标准状态。AI 辅助工具在代码编写上效率最高，而底座环境的创建由开发者通过终端手动完成更为稳妥。

2. 操作步骤

在计算机终端（Terminal）中，依次执行以下命令完成底层脚手架的初始化与依赖组件安装：

```
# 使用 Vite 初始化 Vue 3 基础工程并选择 vue-javascript 模板
npm create vite@latest ai-companion-admin -- --template vue

# 进入项目根目录
cd ai-companion-admin

# 安装 Element Plus、ECharts、Pinia 状态管理、Vue Router 4 路由及 Axios 网络请求库
npm install element-plus echarts pinia vue-router axios @element-plus/icons-vue -save

# 测试运行开发环境，确保初始化成功
npm run dev
```

二、基于 AI 助力的项目目录结构生成

1. 概念引入：项目目录分层设计的意义

管理后台系统往往功能繁多、逻辑复杂。合理的目录结构可以将项目拆分为视图层（Views）、业务服务层（Api）、全局状态管理层（Store）和基础公共组件（Components）。这种划分可以避免写出高耦合的代码，便于多人团队协同开发，同时为后期将数据源从 Mock 切换为真实后端提供便利。

2. 操作步骤

- 在 AI 编辑器中打开刚才创建的 ai-companion-admin 项目文件夹。
- 在 AI 聊天面板中直接发送以下指令，让 AI 为系统创建空目录与对应的核心空白文件。

3. 项目目录构建提示词

请在当前的 `vue 3` 项目的 ``src/`` 目录下，为我自动构建以下分层目录和空白文件，文件内容只需包含基础的 `ES6` 模块导出或占位，以便后续填充：

1. 创建业务服务接口目录 ``src/api/``，包含以下文件：
 - `src/api/auth.js`
 - `src/api/dashboard.js`
 - `src/api/user.js`
 - `src/api/skill.js`
 - `src/api/record.js`
 - `src/api/ai.js`
2. 创建全局状态管理目录 ``src/store/``，包含以下文件：
 - `src/store/app.js`
 - `src/store/user.js`
3. 创建视图组件页面目录 ``src/views/``，包含以下页面文件：
 - `src/views/Login.vue`
 - `src/views/Dashboard.vue`
 - `src/views/User.vue`
 - `src/views/Skill.vue`
 - `src/views/Record.vue`
 - `src/views/AiMonitor.vue`
4. 创建全局公共组件目录 ``src/components/``，包含以下布局文件：
 - `src/components/Layout.vue`

确认这些空目录和文件全部创建无误后，返回成功的确认信息。

三、定义数据模型与异步 Mock 服务

1. 概念引入：什么是异步 Mock 数据？

在前后端并行开发时，后端接口往往尚未部署完毕。前端团队需要模拟一套格式规范的临时数据进行界面渲染，这套数据被称为 **Mock 数据**。

异步 Mock 的核心价值在于：通过 JavaScript 的 Promise 机制，配合 `setTimeout` 定时器模拟网络请求延迟（如延迟 400 毫秒）。这意味着视图层（UI）在调用获取数据时，必须使用标准的 `async/await` 异步语法。第四周进行接口联调时，**开发团队只需重写 API 层内部的代码，而无需改动任何 UI 页面的视图逻辑。**

2. 操作步骤

将以下指令发送给 AI 编辑器，其会自动分析数据流并编写对应的 Mock 服务代码及 Pinia 全局状态。

3. 数据模型与 Mock 服务生成提示词

请根据给定的项目结构，在 `src/api/` 和 `src/store/` 文件夹下生成符合规范的 JavaScript 数据模型和 Mock 服务代码。

编写规范要求：

- 所有接口方法均需返回 Promise 对象，并使用 setTimeout 延迟 400 毫秒返回，以此模拟网络请求延迟。
- 数据逻辑不使用 TypeScript，采用纯 JavaScript。

具体模块代码生成要求：

1. 认证模块（src/api/auth.js 与 src/store/user.js）：

- `auth.js` 导出 `login(username, password)` 方法，接收参数并校验，若匹配 admin/admin123 则返回包含 token 与用户基本信息的成功响应，否则返回 400 错误。
- `user.js`（Pinia Store）维护全局 `token` 和 `userInfo` 状态，并包含 `login` 和 `logout` 动作，token 需持久化至 localStorage。

2. 仪表盘模块（src/api/dashboard.js）：

- 导出 `getDashboardStats()` 方法，返回包含总用户数（1,286）、总技能数（156）、学习记录总数（8,432）、累计调用量（23,567）的统计对象。
- 导出 `getSkillCategoryDistribution()` 方法，返回 5 大核心分类（FRONTEND、BACKEND、DATABASE、DEVOPS、SOFT_SKILLS）的人数占比数据。
- 导出 `getAiCallTrend()` 方法，返回包含最近 7 天日期和对应调用次数的趋势图数据。

3. 用户管理模块（src/api/user.js）：

- 导出 `getUserList(params)` 方法，模拟返回分页的用户列表数据，用户属性包含：id, username, nickname, email, role (ADMIN/TEACHER/STUDENT), status (0-禁用/1-启用), createTime。
- 导出 `updateUserRole(userId, role)` 与 `updateUserStatus(userId, status)` 异步模拟方法。

4. 技能树管理模块（src/api/skill.js）：

- 导出 `getSkillTree()` 方法，返回具有树形嵌套格式的技能树列表数据，每个节点包含 id, name, category, description, children[]。
- 导出技能的增、删、改异步模拟方法。

5. 记录与 AI 监控模块（src/api/record.js, src/api/ai.js）：

- 导出分页查询学习记录与行内删除记录的异步方法。
- 导出 AI 对话测试和发起面试测试的核心 Mock 方法。

四、 页面级组件静态开发

在开发各具体管理视图前，必须保证全系统设计规范与通用组件交互一致。本步骤提供了 6 个独立视图页面的 Qoder/Trae 提示词。

统一设计规范（作为各 Prompt 的约束）

- 主体配色**：主色调科技蓝 #409EFF，背景灰色 #F5F7FA，侧边栏 #304156。
- 状态管理限制**：页面通过服务类提供的接口进行数据载入，使用 Element Plus 进行排版。

1. 登录页 (src/views/Login.vue) 开发提示词

请在 `src/views/Login.vue` 中生成独立的登录界面组件，代码要求遵循以下规范：

样式与视觉规范：

- 背景：采用蓝色的全屏渐变（从左上到右下：#409EFF 到 #66B1FF），无侧边栏布局，独立全屏。
- 主体卡片：宽度 400px，居中放置，纯白背景，圆角 12px，带阴影。
- 文字信息：主标题 "AI伴学平台"（24px 加粗，深色 #303133），副标题 "后台管理系统"（14px 灰色，居中）。

表单与验证规范：

- 使用 `el-form`、`el-form-item` 进行排版。
- 用户名输入框：配有 `el-icon-user` 头部图标。
- 密码输入框：配有 `el-icon-lock` 头部图标，类型为 `password`，支持按回车键登录。
- 表单验证规则：用户名必填，密码必填且字符长度不能少于 6 位。

接口与交互对接：

- 引入全局 `userStore` 状态。
- 点击“登录”按钮触发页面表单校验：
 - 校验中按钮需要置为 `loading` 状态。
 - 校验成功，调用 `userStore.login`。
 - 接口成功后弹出提示并使用 `router.push` 跳转到 `'/dashboard'` 路由，失败则弹出错误提示恢复普通按钮状态。

2. 全局通用框架组件 (src/components/Layout.vue) 开发提示词

请在 `src/components/Layout.vue` 中开发通用的后台框架布局组件，所有管理子视图将挂载在内容区：

布局样式规范：

- 采用 Element Plus 的 `Container` 容器组件。
- 侧边栏 (`Aside`)：
 - 宽度在展开时为 220px，当 `appStore.isCollapse` 为真时收起至 64px。
 - 背景色设为 #304156。顶部显示 "AI伴学管理平台" 字符。
 - 侧边栏菜单 (`el-menu`) 高亮选中态设为蓝色 #409EFF，选项包含：数据总览 (Odometer图标)、用户管理 (User图标)、技能管理 (Share图标)、学习记录 (Document图标)、AI服务监控 (Cpu图标)。
- 右侧内容区：
 - 顶部导航 (`Header`)：高度 60px，带有面包屑导航和用户头像下拉菜单 (`ElDropdown`，包含个人资料与退出登录)。
 - 中部视图 (`Main`)：背景色 #F5F7FA，挂载 `<router-view>` 组件展示子路由页面。

路由导航交互：

- 菜单栏点击触发对应的 `Vue Router` 路由跳转。

- 下拉菜单点击“退出登录”触发 ``userStore.logout``，完成状态清空并回到 ``/login`` 路由。

3. 数据总览页 (src/views/Dashboard.vue) 开发提示词

请在 ``src/views/Dashboard.vue`` 中生成仪表盘看板组件：

看板数据指标与图表布局：

- 顶部数据卡片列：水平排列 4 个指标卡片（带悬浮微移特效）。
 1. 总用户数：1,286，配有蓝色圆形背景图标。
 2. 技能节点总数：156，配有绿色树状图标。
 3. 伴学记录总数：8,432，配有橙色文件图标。
 4. AI 累计调用量：23,567，配有紫色机器人图标。
- 中间统计图表（横向比例分栏为 60% : 40%）：
 - 左侧：环形图。渲染 5 大分类（前端、后端、数据库、DevOps、软技能）的学习分布情况。
 - 右侧：面积折线图。展现近 7 天 AI 调用趋势变化。
- 底部数据列表：使用 ``el-table`` 展示最近 5 条动态（展示字段：用户名称、操作类型、触发时间）。

接口预留逻辑：

- 导入 ``../api/dashboard.js`` 服务。
- 在 ``onMounted`` 钩子函数中，异步读取数据：
 1. 调用 ``getDashboardStats()`` 刷新顶部 4 个统计数据。
 2. 调用 ``getSkillCategoryDistribution()`` 和 ``getAiCallTrend()``。获取成功后初始化 ECharts。
 3. 注意在生命周期销毁（`onUnmounted`）中，释放 ECharts 图表实例。
 4. 读数过程中必须包含全屏或局部卡片 `el-loading` 效果。

4. 用户管理页 (src/views/User.vue) 开发提示词

请在 ``src/views/User.vue`` 中生成用户管理功能视图，要求如下：

数据查询与筛选栏：

- 搜索卡片居顶：包含用户名模糊输入框、角色下拉框（全部/管理员/教师/学生）、搜索按钮（配搜索图标）、重置按钮。

表格列定义与配置（`el-table`）：

- 字段列：用户ID、用户名、登录昵称、安全邮箱、用户角色（角色渲染：admin展示红色Tag，teacher展示蓝色Tag，student展示绿色Tag）、注册时间。
- 行内修改操作：提供状态开关（`el-switch`）在线控制禁用/启用状态，以及一个行内的下拉框直接修改该用户的所属角色。
- 底部配有标准的 `el-pagination` 分页器，支持每页条数切换。

逻辑控制与接口联调预留：

- 引入 ``../api/user.js`` 中 ``getUserList``、``updateUserStatus`` 和 ``updateUserRole`` 的接口方法。
- 页面加载触发 ``getUserList`` 拉取数据并刷新表格。
- 点击搜索重置当前页码并传入参数重新拉取。
- 行内 Switch 开关改变状态：自动发出异步修改请求，成功后给出顶部提示。

- 行内角色 **Selector** 改变：弹出 **Element Plus** 确认对话框，用户确认后再向后端发异步请求。若点击取消，回滚表格选择值。

5. 技能树管理页 (src/views/Skill.vue) 开发提示词

请在 `src/views/Skill.vue` 中生成树形结构技能维护面板：

工具栏与组件排版：

- 操作栏居顶：配有分类下拉框（**FRONTEND/BACKEND/DATABASE/DEVOPS/SOFT_SKILLS**）和 "新增根技能" 按钮（绿色，带加号图标）。
- 技能树主体：使用 **Element Plus** 的 `el-tree` 组件，节点支持缩进和层级收展。
 - 自定义节点渲染：左边显示技能名字与对应分类的圆角小胶囊标签，右边显示两个文字按钮：“编辑”（蓝色）和“删除”（红色）。

表单与弹窗控制：

- 增删改共用一个 **el-dialog**：表单项包含技能名字（必填）、归属技能分类（必填，下拉单选）、技能简述（文本域）、父级技能选择器（下拉树或普通下拉）。
- 点击删除：触发确认对话框，告知“删除此技能将级联清除其所有的子技能分支，是否确定删除？”。

逻辑控制与接口对接：

- 引入 `../api/skill.js` 的 **Mock** 服务。
- 组件生命周期里抓取技能列表，转化结构赋值给 **el-tree** 数据属性。
- 弹窗提交前，需对必填表单进行校验。接口调用时设置提交按钮 `loading` 属性防止二次触发，调用成功关闭弹窗并刷新技能列表。

6. 学习记录页 (src/views/Record.vue) 开发提示词

请在 `src/views/Record.vue` 中生成伴学学习笔记的日常运维列表页面：

过滤搜索栏：

- 提供按用户 **ID** 搜索的输入框、技能关键字模糊查询输入框，查询和重置按钮。

数据表格配置：

- 表格列定义：记录**ID**、学习者用户**ID**、技能名称、学习日记正文（由于可能超长，设置溢出省略属性，鼠标悬停时使用 **el-tooltip** 浮出显示全部文字内容）、学习耗时（分钟，带时钟图标）、创建时间。
- 操作列：包含“删除”字样的红色文字按钮。
- 底部分页区：包含标准的 **el-pagination** 控件。

接口与联调逻辑预留：

- 引入 `../api/record.js` 服务层。
- 定义分页大小、当前页和搜索条件的绑定状态，触发翻页时在底层传参刷新列表。
- 点击“删除记录”时弹出确认警告对话框，确认后异步请求后端删除接口，并在前端刷新当前列表。

7. AI 服务监控页 (src/views/AiMonitor.vue) 开发提示词

请在 `src/views/AiMonitor.vue` 中开发 AI 会话沙箱和全员面试结果统计面板：

分栏排版设计：

采用横向对称的分栏比例设计（55% 宽度 ： 45% 宽度，中间预留 16px 间距）：

- 左侧面板：AI对话模拟沙箱。
 - 顶部配有测试标题和绿色的在线呼吸灯。
 - 中间对话消息区（高度 400px，设为可滚动，浅灰色背景）：气泡流渲染。用户发送的消息靠右展示（蓝色气泡，白字）；AI 返回的消息靠左展示（白色气泡，黑字），每一条都带上时间。
 - 底部配有文本输入框与“发送测试消息”按钮。
- 右侧面板（上下排布）：
 - 右上：模拟面试在线触发沙箱。输入技能 ID 点击“开始模拟测试”，模拟出题与答题，最后在卡片展示面试评估的分数和用时。
 - 右下：面试历史运行记录列表。展示最近 5 条全员面试结果（用户、目标岗位、状态标签、面试得分）。每一行带一个“查看详情”按钮。

接口逻辑预留：

- 输入框输入文本点击发送后，列表本地追加一条用户气泡，同时渲染一个“AI正在检索解析...”的骨架屏气泡。异步发送请求得到 AI 数据后，用真实的 AI 回复替换骨架屏。
- 历史列表单行点击“查看详情”时，弹出 el-dialog，在弹窗中渲染本轮面试的 Markdown 诊断报告。

五、 演示：UI 组件调用 Mock 服务（操作指引）

当把以上的 API 接口文件和 6 个 .vue 页面生成完毕后，可以参照以下操作，演示和检验 UI 组件是如何通过异步机制与 Mock 服务对接的。

1. 数据流动原理（以 Dashboard.vue 为例）

[Dashboard 页面挂载]

└─

▼ 1. 触发 onMounted 生命周期

[加载状态启动] ──> 页面 el-loading 设置为 true，展示加载状态

└─

▼ 2. 异步请求：await getDashboardStats()

[api/dashboard.js] ──> 模拟 400ms 网络延迟后，返回 mock 数据

└─

▼ 3. 收到 Promise 返回值

[页面状态更新] ──> 将 `data.totalUsers` 等数值更新到页面 @State 中

└─

▼ 4. 刷新完毕

[加载状态关闭] ──> el-loading 设置为 false，图表载入，页面渲染呈现

2. 核心对接代码（可直接阅读并排查）

打开 src/views/Dashboard.vue，核心的异步消费调用方法应严格遵循如下规范书写：

```
<template>
  <!-- 当 loading 为 true 时，卡片展示加载中状态 -->
  <div v-loading="loading" class="dashboard-container">
```

```

<!-- 指标卡片行 -->
<el-row :gutter="16">
  <el-col :span="6">
    <el-card shadow="hover">
      <div class="card-item">
        <span class="card-value">{{ stats.totalUsers }}</span>
        <span class="card-label">总用户数</span>
      </div>
    </el-card>
  </el-col>
  <!-- 其余三张卡片省略... -->
</el-row>

<!-- 图表渲染区 -->
<el-row :gutter="16" class="chart-row">
  <el-col :span="14">
    <el-card header="技能分类分布">
      <div ref="pieChartRef" style="height: 350px;"></div>
    </el-card>
  </el-col>
</el-row>
</div>
</template>

<script setup>
import { ref, onMounted, onUnmounted } from 'vue';
import * as echarts from 'echarts';
import { getDashboardStats, getSkillCategoryDistribution } from
'../api/dashboard';

// 定义双向响应状态
const loading = ref(false);
const stats = ref({ totalUsers: 0, totalSkills: 0, totalChats: 0,
totalInterviews: 0 });
const pieChartRef = ref(null);
let myChart = null;

// 页面加载时的生命周期函数
onMounted(async () => {
  loading.value = true;
  try {
    // 1. 发起异步接口请求, 使用 await 同步化代码
    const statsRes = await getDashboardStats();
    stats.value = statsRes.data;

    // 2. 加载图表数据并渲染
    const chartRes = await getSkillCategoryDistribution();
    initChart(chartRes.data);
  } catch (error) {
    console.error("加载数据大屏失败", error);
  } finally {
    loading.value = false; // 结束等待动画
  }
});

// 初始化 ECharts 方法

```



```

const initChart = (chartData) => {
  if (pieChartRef.value) {
    myChart = echarts.init(pieChartRef.value);
    const option = {
      tooltip: { trigger: 'item' },
      series: [
        {
          name: '技能分类',
          type: 'pie',
          radius: ['40%', '70%'],
          data: chartData // 注入异步返回的数据
        }
      ]
    };
    myChart.setOption(option);
  }
};

// 页面销毁生命周期，防止内存泄漏
onUnmounted(() => {
  if (myChart) {
    myChart.dispose();
  }
});
</script>

```

3. 如何切换为真实后端数据源？

当后端的业务微服务开发就绪后，开发团队**不需要改动任何 views/Dashboard.vue 的代码**，只需打开 src/api/dashboard.js，将原有的异步 Mock 函数修改为使用 axios 向网关或单体服务发送真实网络连接即可：

```

// 导入封装了全局请求头拦截器（附带 JWT Token）的 Axios 实例
import request from '../utils/request';

// 联调期修改：直接用 Axios 请求替换 Mock
export const getDashboardStats = async () => {
  // 向真实后端发起 GET 请求获取指标
  const response = await request.get('/dashboard/stats');
  return response.data; // 只要后端返回的数据 JSON 结构不变，前端页面将无缝运行
};

```

这一模式可以将静态页面的快速编写与复杂的系统对接完全剥离，也是前端工程化实践中的核心规范流程。